

ZenTrade

An autonomous senior trader for
the Indian equity market.

Paper simulation only

NSE equities

Go · TypeScript · Python

Executive overview

Most automated trading systems are rule engines: a condition fires, an order goes out. They cannot explain themselves, they cannot notice that their reason for holding a position has quietly stopped being true, and they cannot argue with themselves.

ZenTrade is built around a different premise — that the valuable part of a discretionary trader is not the entry signal but the **continuous reassessment**. A senior trader forms a view, writes down what would prove it wrong, watches the market, and changes their mind when the evidence changes. ZenTrade does that, autonomously, and shows its work.

Two things make it more than a language model with a broker attached. First, **deterministic code owns every safety decision** — risk, sizing, legality, arithmetic, and permission to execute. The model contributes interpretation and nothing else. Second, **protection never waits for reasoning**: a stop breach is acted on in microseconds by a separate process that has no model client linked into it.

It trades paper only. There is no order-placement code anywhere in the repository — the safety boundary is absence, not a setting.

The two goals

GOAL 1 — A SENIOR-LEVEL AUTONOMOUS TRADING AGENT

Not "the agent can output BUY, SELL or HOLD". The bar is behavioural: it maintains a live model of the market, notices when something material changes, forms a thesis, attacks its own thesis, considers what else could explain what it sees, prices the opportunity against real costs, checks whether the world moved while it was thinking, and keeps reassessing every position after every meaningful change.

GOAL 2 — THE RIGHT LANGUAGE FOR EACH WORKLOAD

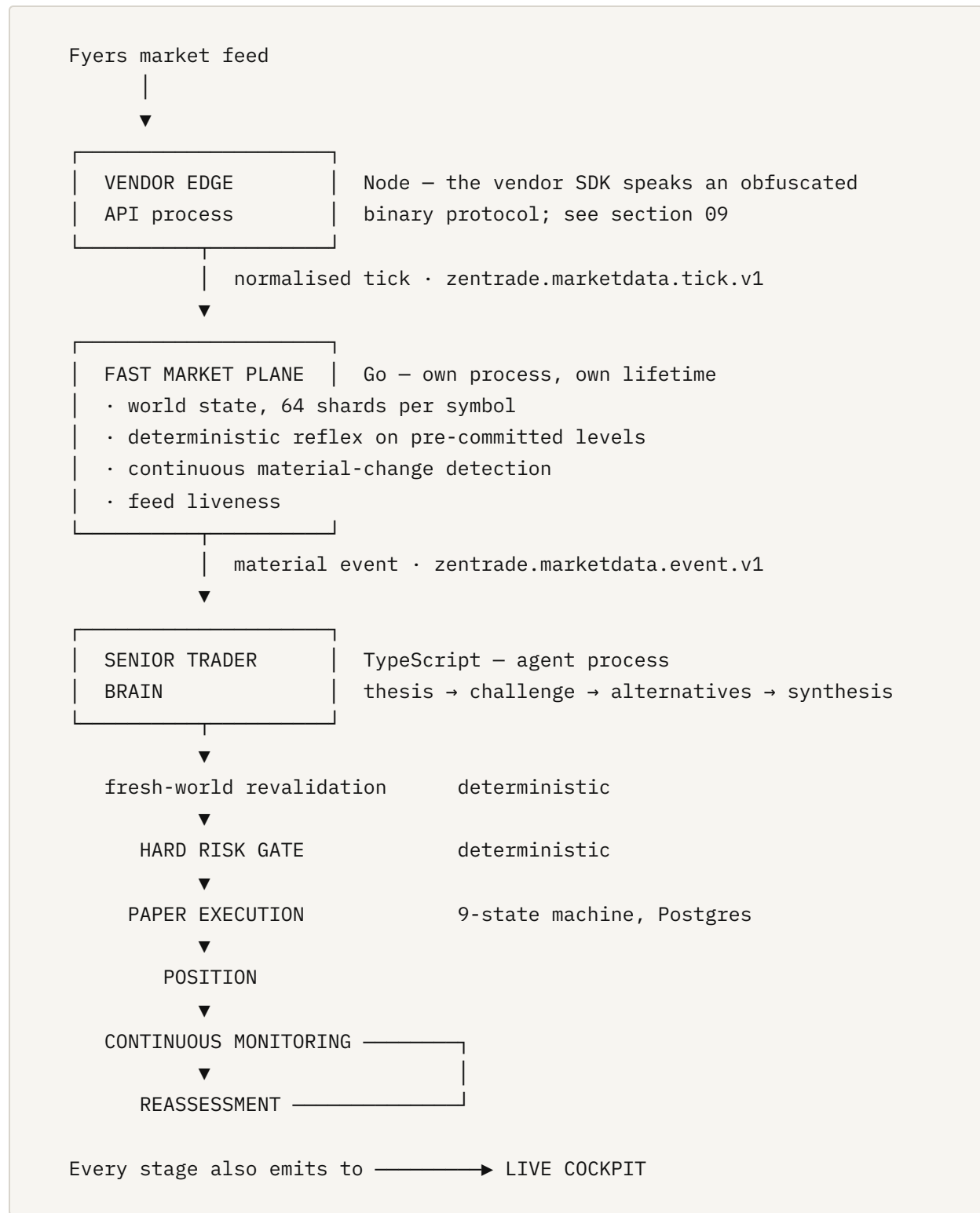
Not "rewrite it in Go". The instruction was to migrate *only* where there is a measured or structural reason, and to say so plainly where there is not. That discipline is why one component moved, one was rejected on a vendor constraint, one was fixed rather than migrated, and C++ was rejected outright with the numbers to justify it.

How the trader thinks

observe	continuous, every tick
understand	what is FACT, what is inference
form thesis	a falsifiable view, with its own kill conditions
challenge	a second pass whose job is to break the first
alternatives	what else explains the same observation?
synthesise	deterministic: R:R, cost hurdle, EV, opportunity cost
revalidate	has the world moved while I was thinking?
risk gate	deterministic, non-negotiable
act	paper execution
reassess	after every material change, forever

Each stage is a real artifact the system produces and records — not a narration written for display. The cockpit shows exactly these objects.

System architecture



Three processes, three responsibilities, no overlap. The backend owns the vendor edge and the cockpit's transport. The agent owns the trading runtime. The fast plane

owns the tick. None of them can do another's job.

05

The life of one tick

A single price update from the exchange:

1. **Normalised at the edge.** The vendor's float rupee value becomes integer paise and dies at this boundary. Timestamps become epoch milliseconds; receipt time and exchange time stay separate fields.
2. **Published** onto Redis, where both the fast plane and the agent are listening.
3. **The fast plane updates the world.** Last, high, low and a monotonic per-symbol sequence, in the shard that symbol hashes to — **28.99 ns, zero allocations.**
4. **Levels are tested.** If the tick crosses a stop, a target or a named invalidation the thesis pre-committed to, an event is emitted immediately.
5. **Material change is detected.** Approach bands, price velocity over a real window, VWAP deviation, and volume inside the minute in progress.
6. **Nothing happens** if none of that is true — which is the common case, and the system stays silent rather than manufacturing activity.

Emitted events reach the brain by push, not by poll: roughly a millisecond from the tick, not a drain interval.

The fast plane, and why it is Go

The honest reason is **not speed**. Node evaluates the same reflex in 83 nanoseconds, which is fast enough. On the real detection workload Go is 1.24× faster — not a reason to migrate anything.

The reason is **isolation**. Tick receipt and protection used to share an event loop with model orchestration and HTTP serving, and a measured **86.8 ms of head-of-line blocking** proved that coupling was real. They also shared a process lifetime, and the backend exits on any uncaught exception — so an unrelated bug dropped the market feed mid-session.

Why a language boundary rather than just a second Node process. A minimal Node process would isolate it today and drift back into a brain within a quarter, because adding an import is free. Across a language boundary, adding a model client to the deterministic safety path is not a temptation — it is a rewrite. The fast plane has no model client, no database driver and no order path linked into it, and cannot acquire one by accident.

MEASUREMENT	BEFORE	AFTER
Tick → reflex, at a bar boundary	86.844 ms p50	0.047 ms p50
Parallel, 200 symbols	122.1 ns/op	25.62 ns/op
Protective path, serial	—	28.99 ns/op, 0 allocations
Cross-language parity	—	100,000 ticks, 848 events, zero divergence

The Senior Trader Brain, and why it stays TypeScript

The reasoning pipeline is **bound by the model, not by the language**. Two sequential model calls dominate at seconds each; the orchestration around them is nanoseconds. Moving it to Go would make it no faster to run and considerably slower to change — and it is the part of the system most likely to change weekly.

The risk gate stays with it deliberately. Splitting a decision from the gate that authorises it across a process boundary adds a failure mode and buys nothing.

The research plane, and why it stays Python

Point-in-time discipline, a 75.8-million-row bar spine, feature engineering, calibration, replay and backtesting all run on Polars and DuckDB — which are Rust and C++ underneath. Writing our own would be slower and less correct.

The research protocol is frozen and was not touched: 156 trials, a Bonferroni threshold of 3.178, and **zero holdout looks**. Rewriting the harness would put those numbers at risk for no gain.

The C++ decision

No C++ was written, and that is the finding.

At a generous 1,000 ticks per second the entire fast plane consumes **0.0083 % of one core**. The reflex is **0.0056 %** of the protective path it feeds; the remaining 99.99 % is Postgres round trips against an instance whose bare `SELECT 1` measures 322.6 ms. Optimising the reflex in C++ would be the wrong end of the problem by five orders of magnitude.

There is also nothing to compute. The system consumes last-traded price and cumulative volume. No spread, no book imbalance, no queue position — microstructure code with no microstructure data.

Where C++ would earn its place. Level-2 depth: 200 symbols × 20 levels × 2 sides, roughly 100× the message rate, with every message mutating a book rather than comparing a scalar. The boundary is pre-drawn behind the same frozen contract, with one precondition — build it in Go first and profile to saturation. If Go holds, the answer stays no. No speculative code exists.

A VENDOR CONSTRAINT WORTH NAMING

The Fyers market-data socket speaks a proprietary binary protocol. The vendor SDK ships it **fully obfuscated** with no published schema. Reimplementing it in Go or C++ would mean reverse-engineering obfuscated code against a venue that cannot be exercised outside market hours. That is why the vendor edge stays in Node — a constraint, not a preference.

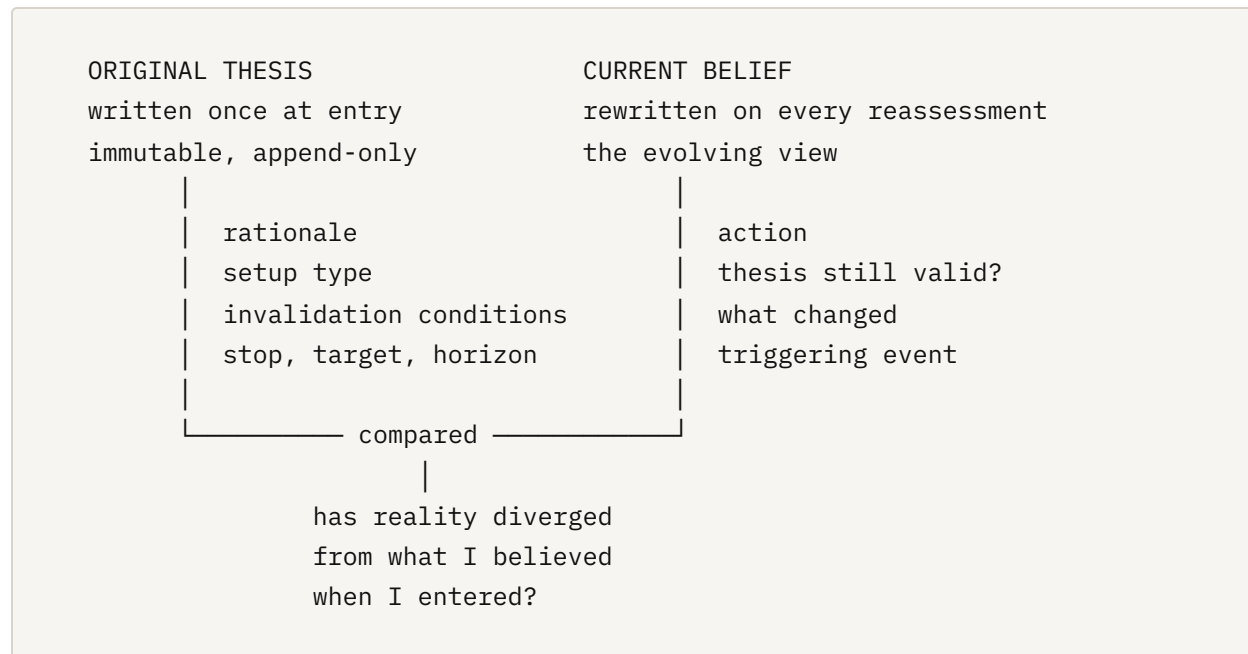
Evidence hierarchy

A senior trader does not confuse "price is above VWAP" with "buyers are aggressive". The first is arithmetic; the second is an interpretation that may be wrong. Conflating them is how a weak inference acquires the authority of a measurement.

TIER	MEANING	ASSIGNED BY
FACT	Measured or computed from data held	deterministic code
OBSERVATION	A measured comparison against a baseline	deterministic code
INFERENCE	A conclusion drawn from facts	the model — its ceiling
HYPOTHESIS	A candidate explanation, not established	the model
PREDICTION	A claim about the future	the model

Tiers are assigned by origin, never claimed. Anything the model produces is clamped to INFERENCE or weaker. It cannot promote its own reasoning to fact.

Thesis architecture



Keeping these apart is the whole point. A system that edits its entry thesis as the market moves cannot tell you it was wrong — it can only tell you what it currently believes, which is worth much less. The cockpit shows them side by side, and the entry thesis is enforced immutable at the database level.

Adversarial reasoning

Forming a thesis is easy; a language model will produce one for any chart. The value is in the second pass, whose only job is to break the first.

ARTIFACT	WHAT IT ASKS
Strongest objection	What is the single best argument against this?
Counter-thesis	What would I trade <i>instead</i> , stated as a position?
Alternative hypotheses	Short covering? Index beta? Thin liquidity? A single large order?
Disconfirming evidence	What in the data contradicts this view?
What would change my mind	Named, falsifiable conditions
Confirmation bias check	Was the author only looking for supporting evidence?

The challenger runs at near-zero temperature, so "this thesis is weak" is a stable judgement rather than a sample. It can only ever make the decision **more conservative**: a broken thesis cannot become a buy, and a weak one cannot open new exposure. And a thesis with no invalidation condition is not a thesis — the action is forced to HOLD.

Deterministic safety

Everything below is computed in code the model cannot reach or influence.

CONTROL	WHAT IT ENFORCES
Risk–reward	Computed from the levels, not from the model's claim about them
Cost hurdle	73.55 bps round trip. An edge that does not clear it is not an edge
Expected value	Derived from R:R and the cost hurdle, never asserted
Opportunity cost	Is this better than doing nothing?
Position sizing	Deterministic. A BUY with no usable size degrades to HOLD
Session policy	No new exposure into the close; exits stay possible
Freshness	Stale data blocks new exposure automatically
Single writer	One module writes cash, positions and orders — enforced by test
Idempotency	Intent-derived order keys; a repeat is absorbed, not duplicated

Continuous observation

The system used to poll every fifteen seconds. That is not a tuning problem, it is a category error: a poll samples endpoints and cannot see what happened between them.

A price that moves 3.5% and returns within 14 seconds:

15-second poll	t=0 ₹1,000	t=15 ₹1,000
	"nothing happened"	
continuous	t=0 ₹1,000	
	t=7 ₹1,035	← detected on this tick
	t=14 ₹1,000	

Eight conditions are now evaluated on every tick: stop, invalidation, target, stop approach, target approach, price velocity, VWAP deviation and volume. Only feed staleness remains timer-driven, and necessarily — the absence of ticks cannot arrive as a tick.

The supervisory scheduler still runs, but for what schedulers are for: reconciliation, order expiry, health and end-of-day. It is no longer how the system finds out that the market moved.

Protective reflex

The governing idea: **the line between the fast path and the slow path is not drawn by latency budget. It is drawn by who already made the decision.**

A thesis is a pre-commitment. It named its stop, its target and the price that would prove it wrong, and the risk gate already authorised the position those levels belong to. When a tick crosses one of them there is no judgement left to make — so there is nothing for a model to decide.

	BEFORE	NOW
Stop breach noticed	15-second poll	the tick that causes it
Then	queued behind two model calls	acted on immediately
Level lost → order exists	≈ 35 seconds	≈ 1 millisecond
Model consulted	yes	no

Reasoning still happens — afterwards, to decide what the crossing *meant*, not whether to protect. And only a crossing of a pre-committed level authorises action; everything else is an attention signal that wakes the trader without moving the position.

Memory and learning

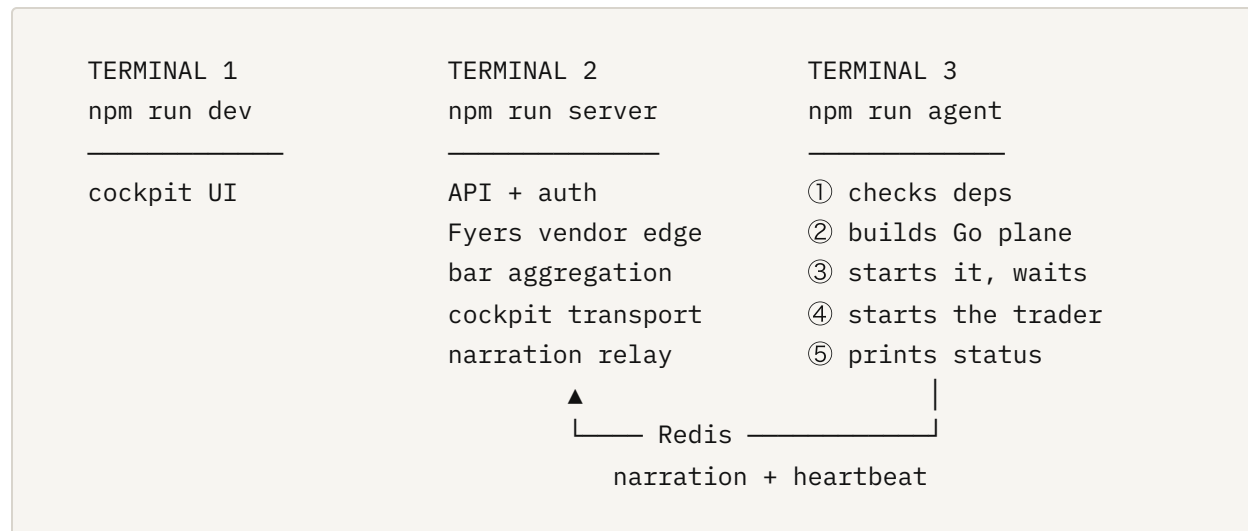
trade	→	outcome	→	reassessment	→	calibration	→	future context
journal		labelled		evolving		how well		retrieved as
entry		at a		belief		calibrated		OBSERVATION on
		horizon				was I?		the next decision

Every decision is journalled with its evidence. Outcomes are labelled with strict no-look-ahead rules. Episodic memories index that experience, and retrieval surfaces comparable past decisions when a similar one is being made.

Two rules keep this honest. Retrieved memories enter as **OBSERVATION**, never as current market fact, and contradictory episodes surface together rather than the flattering one being shown alone. Eligibility is strictly before the deciding session, so a memory can never contain the outcome of the decision it is informing.

No alpha is claimed from this. It is context, and it is labelled as context.

The autonomous runtime

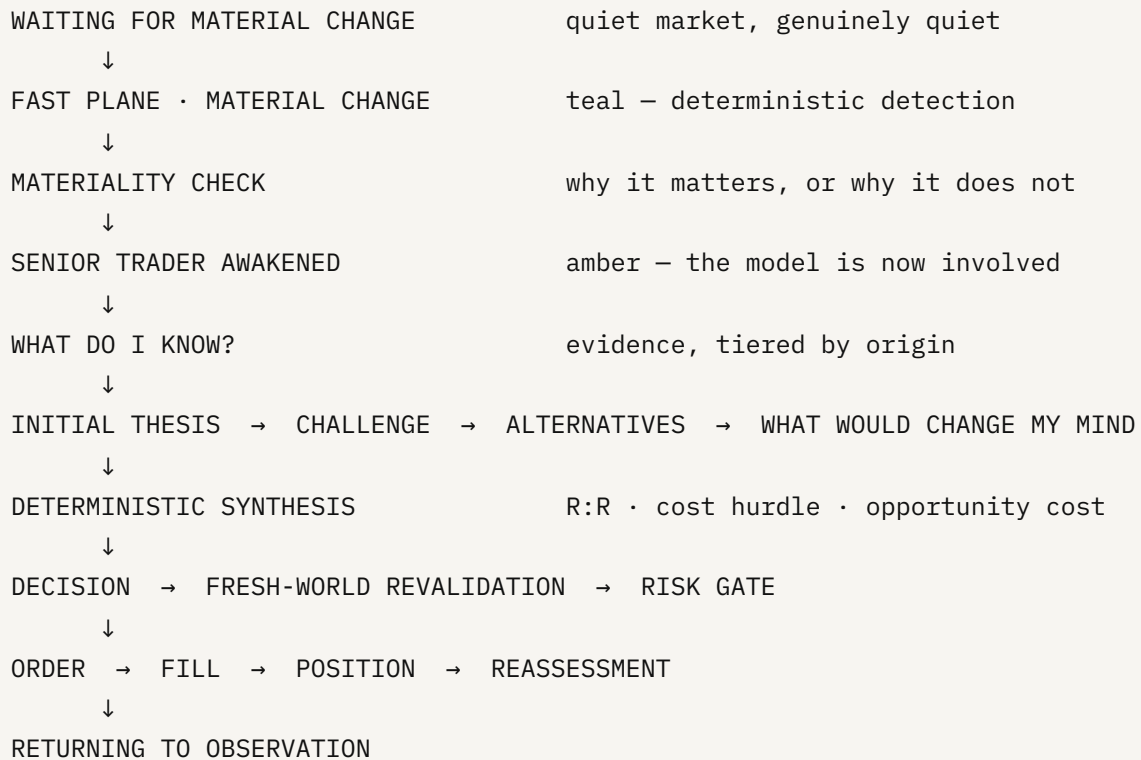


Exactly one autonomous runtime exists, and it lives in the agent process. The backend cannot start a second one — the code to do so does not exist there, and a test asserts it.

The split earns its keep: the trader restarts without dropping the market feed, and the cockpit reloads without disturbing the trader. The agent writes a short-TTL heartbeat, so the cockpit distinguishes a quiet trader from a stopped one and says **TRADER NOT RUNNING** rather than reporting ARMED.

The live cockpit

Not a dashboard. The intent is the experience of sitting beside an autonomous trader while it works.



The rule the whole interface rests on: **nothing generates activity**. There is no timer, no synthetic event, no thinking animation. When the market is quiet the screen is quiet and says so. A value the system does not have renders as `UNKNOWN` ; confidence with no stated basis renders as `INSUFFICIENT BASIS` .

The browser is read-only by construction — only GET endpoints, no import that can reach execution, and a test that fires every mutating verb at the running server to prove it.

A decision, end to end

```

09:47:12 FAST PLANE      RELIANCE – volume 3.7× the 60-bar median
09:47:12 MATERIALITY    WARNING on a position route → reasoning required
09:47:12 BRAIN AWAKENED  trigger VOLUME_SPIKE

09:47:13 WHAT I KNOW    [FACT]          price ₹1,425
                        [FACT]          session VWAP ₹1,401
                        [OBSERVATION] volume 3.7× baseline
                        [OBSERVATION] 2 comparable past decisions: 1 stop, 1
entry thesis (immutable): "VWAP reclaim, 09:20"

09:47:14 THESIS        continuation above VWAP on expanding participation
                        contradicting: volume rising while breadth is flat
                        invalidated by: a 5m close below VWAP

09:47:15 CHALLENGE      verdict THESIS_WEAK
                        objection: breadth does not confirm the move
                        counter-thesis: thin-liquidity drift that mean-reverts
                        alternatives: index beta · one large working order
                        would change my mind: VWAP reclaimed on rising volume

09:47:16 SYNTHESIS      R:R 2.40 · cost hurdle 73.55 bps
                        edge INSUFFICIENT after costs
                        opportunity cost: capital better left unallocated

09:47:16 DECISION       HOLD – confidence LOW
                        basis: challenger judged the thesis weak

09:47:16 NO ORDER       nothing reached the risk gate

09:47:17 RETURNING TO  OBSERVATION

```

This is the ordinary case, and it is the point. A system that only shows you its trades is hiding the decisions that mattered most — the ones where it declined.

Technology ownership

COMPONENT	LANGUAGE	WHY
Fyers vendor socket	Node	obfuscated vendor protocol, no schema
Tick normalisation	Node	must sit with the SDK that decodes the frame
Live world state	Go	per-symbol concurrent, zero-allocation
Deterministic reflex	Go	must not share a lifetime with the brain
Material-change detection	Go	same plane as the state it reads
Feed liveness	Go	supervises the plane it owns
Reasoning pipeline	TypeScript	model-bound; changes weekly
Risk gate	TypeScript	must sit with the decision it gates
Execution & ledger	TypeScript	Postgres-bound
API, cockpit transport	TypeScript	orchestration and fan-out
PIT, spine, features	Python	Polars and DuckDB are already native
Training, replay, backtest	Python	ecosystem is the reason to be there
Order-book engine	C++ – future	not justified by any current workload

Why the model cannot trade

The model is one voice in a system that does not take its word for anything.

model proposes	→ deterministic validation	illegal action → HOLD
	→ adversarial challenge	can only be more conservative
	→ deterministic synthesis	arithmetic, never asserted
	→ fresh-world revalidation	is this still true?
	→ hard risk gate	deterministic, non-negotiable
	→ execution	9-state machine, idempotent

The model never determines risk, legality, quantity, arithmetic, freshness or permission to execute. It cannot reach the execution engine — there is no path from the reasoning modules to it. A malformed response becomes HOLD. A model outage becomes HOLD. And the protective path, where a wrong answer is most expensive, does not consult it at all.

Engineering progress

CODE

SUITE	RESULT
TypeScript / JavaScript	1,076 passing
Cockpit (React)	50 passing
Go	all packages, race detector clean
Python research	557 passing
Cross-language parity	20 golden fixtures, zero divergence
Shadow replay	100,000 ticks · 848 events · zero divergence

The complete path is verified end to end against the **real compiled Go binary in its own process**: a tick published to Redis, detected by the plane, delivered to the brain, and closing a real position through the real execution engine — with the cockpit showing it, attributed to the plane that detected it.

LIVE VALIDATION

Separate, and honestly reported: **the system has never processed a tick from a live market session**. Everything above is proven against a real database, a real cache, the real Go binary and the real reasoning pipeline — but not against a real exchange feed.

Remaining limitations

These are external conditions, not unfinished engineering.

LIMITATION	WHAT IT REQUIRES
No live session has run	a trading day with a valid broker token
Fast plane runs in shadow	one clean live session before it becomes authoritative
No measured edge	many sessions of recorded outcomes
Long-only	a short-side money model; deliberately closed and pinned by test
Paper only	intentional — no order-placement code exists

No profitability is claimed. No trading edge is claimed. No live validation is claimed.

Starting it tomorrow

1 Re-authenticate the broker

The token expires at 03:00 IST daily. Then run the preflight — it ends in READY or names the reason.

```
cd apps/api && npm run preflight
```

2 Start the cockpit

```
cd apps/web && npm run dev
```

3 Start the backend

API, vendor edge, cockpit transport. This process does not trade.

```
cd apps/api && npm run server
```

4 Start the trader

Builds and starts the Go fast plane, then the autonomous runtime.

```
cd apps/api && npm run agent
```

Open `http://localhost:5173/trader` and watch. Stop with `Ctrl-C` in the agent terminal: it drains, reconciles, releases the market-data lease and exits cleanly.

Status

Code completion against the approved architecture. Live-market validation is reported separately in section 22 and is not deducted here.

Fast market plane	97%
Senior Trader Brain	93%
Autonomous runtime	97%
Live trader cockpit	96%
Goal 1 — autonomous senior trader	95%
Goal 2 — selective technology architecture	95%
Whole approved architecture	82%

The last figure is held down by seven research milestones — knowledge, skills and evaluation — that are scoped but not started. Nothing on the trading path is waiting on engineering work.

ZenTrade — autonomous senior-trader architecture. Paper simulation only; no order-placement code exists in the system. No profitability, trading edge or live-market validation is claimed.